

OOP

UNIT 2 – FROM STRUCT TO CLASS

"If struct can group related data, class teaches that data must also be protected."

WHY STRUCT IS NOT ENOUGH

Imagine that you are developing a small music application. The application needs to store information about songs: the title, the artist, the duration, and the number of times the song has been played.

A natural way to group related information is by using a struct:

```
struct Song
{
    string title;
    string artist;
    int duration;
    int plays;
};
```

At first glance, this seems perfectly reasonable.

However, consider the following code:

```
Song s;  
s.title = "";  
s.duration = -40;  
s.plays = -7;
```

The program compiles successfully, but the resulting object is clearly **invalid**. A song should not have:

- An empty title
- A negative duration
- A negative number of plays

The problem is that a struct simply groups data together; it does not protect that data from being assigned invalid values.

This raises an important question:

HOW CAN WE ENSURE THAT OBJECTS ALWAYS REMAIN IN A VALID STATE?

In this unit, we take the first practical step toward object-oriented design. We begin with something familiar—struct. Then we explore its limitations and discover why merely grouping data is not enough. Finally, we introduce **classes**, which allow us to combine **data**, **behavior**, and **protection**, giving us greater control over how objects are created and used.

LEARNING OBJECTIVES OF UNIT2

- Explain the limitations of using struct and reason C++ introduced class.
- Explain pass-by-value and pass-by-reference.
- Distinguish between a class and an object.
- Identify data members and member functions.
- Use public and private access specifiers.
- Explain the basic idea of encapsulation.
- Understand why data and behavior should belong together.
- Organize a simple class using .h and .cpp files.

BEFORE WE BEGIN

Before starting this unit, you should already understand:

- struct
- variables and data types
- functions
- passing parameters to functions
- arrays
- basic input and output
- compiling simple C++ programs

You do not need to know classes yet.

That is exactly what we are about to begin.

WHAT YOU KNOW ABOUT STRUCT

A struct allows us to group related information. For example:

```
struct Song
{
    string title;
    string artist;
    int duration;
    int plays;
};
```

Now we can create song variables.

```
Song song1; Song song2; Song song3;
```

Each variable represents a different song.

We can assign values:

```
song1.title = "Imagine";
song1.artist = "John Lennon";
song1.duration = 183;
song1.plays = 1200;
```

This is simple and useful. Instead of managing four separate variables, we now manage one object that represents a song.

THE PROBLEM WITH STRUCT

The problem is that all fields in a struct are public by default.

That means any part of the program can change them directly.

```
song1.duration = -50;  
song1.title = "";  
song1.plays = -100;
```

The compiler does not complain. From the compiler's point of view, these are legal assignments. But from the point of view of our program, they are wrong.

A song with a negative duration is not a valid song. A song with no title is probably not valid either. The structure stores data, but it does not protect it.

WHY IS THIS A SERIOUS PROBLEM?

In a very small program, this may not seem important.

If the program has only one file and one programmer, maybe you can remember not to assign bad values. But real software is different. Real software may contain:

- many files,
- many functions,
- many programmers,
- many years of changes

If every part of the program can directly change the internal data of a song, then every part of the program can also “damage” it.

This makes the program harder to understand and harder to maintain.

Suppose your music application contains 200 functions.

Suppose all of them can directly modify duration.

Later, you discover a bug: some songs have negative durations.

Where did the bug come from? How long will it take to find?

THE PROCEDURAL SOLUTION

One possible solution is to stop changing the fields directly and write functions instead.

```
void printSong(Song song) {  
    cout << song.title << " - " << song.artist << endl;  
}  
void playSong(Song* song) {  
    song->plays++;  
}  
void setDuration(Song* song, int duration) {  
    if (duration > 0)  
        song->duration = duration;  
// The function setDuration can check validity of the value  
}
```

This is better.

Now we can write:

```
setDuration(&song1, 183);  
playSong(&song1);  
printSong(song1);
```

AN IMPORTANT OBSERVATION

All these functions work with Song objects:

```
playSong()  
setDuration()  
printSong()
```

They describe what a song can **do**. But they are written **outside** the Song definition.

KEY QUESTION

If these functions always work on songs, why are they separate from the song itself?

OBJECT-ORIENTED PROGRAMMING BASIC UNIT: CLASS

- Data (**attributes**)
- Functions (**behavior**)

into a single unit called a **class**.

A class keeps related data and behavior together and provides better protection for the object's state.

A SMALL DETOUR: PASSING PARAMETERS TO FUNCTIONS

We just wrote functions that operated on a Song.

```
void playSong(Song song) { //pass by value
    song.plays++;
}
```

Although this looks correct, the function receives a copy of the Song object.

```
Song song1;
song1.plays = 5;

playSong(song1);

cout << song1.plays;    // Output: 5
```

So this can be solved by

```
void playSong(Song* song) { //pass an address of a variable
    song->plays++;
}

playSong(&song1);    // Have to pass address of the variable
```

PASSING PARAMETERS TO FUNCTIONS BY VALUE / BY POINTER

There are three common ways to pass an parameters to a function.

- **Pass by value** – the function receives a copy of the object.
- **Pass by pointer** – the function receives the object's memory address and can access or modify the original object using the -> operator.
- **Pass by reference** – the function receives another name for the original object, allowing direct access without copying.

PASSING BY REFERENCE

```
void playSong(Song& song)
{
    song.plays++;
}
```

The '&' indicates that the parameter is a reference. A reference is another name for an existing object rather than a copy.

```
Song song1;
song1.plays = 5;
playSong(song1);
cout << song1.plays;    // Output: 6
```

WHAT IS A REFERENCE?

A **reference** does not create a new object. Instead, it provides another name for an existing object.

Any modification made through the reference changes the original object.

WHY USE REFERENCES?

- Avoid unnecessary copies of large objects.
- Allow functions to modify the original object.
- Improve program efficiency.

PASSING BY CONST REFERENCE

```
void printSong(const Song& song)
{
    cout << song.title << endl;
}
```

When a function only needs to read an object, passing by const reference *avoids copying* while *guaranteeing that the object cannot be modified*.

AND BACK TO OOP, WE SPOKE ABOUT CLASSES

DATA AND BEHAVIOR BELONG TOGETHER

A song has **information**.

- title
- artist
- duration
- number of plays

A song also has **behavior**.

- print itself
- play
- increase its play count
- update its duration

The data and the behavior describe the same concept.

Object-Oriented Programming says:

Data and behavior that belong together should be placed together.

This is the idea behind a class.

FROM STRUCT TO CLASS

A class allows us to define both the information and the behavior of an object.

```
class Song
{
public:
    string title;
    string artist;
    int duration;
    int plays;

    void print()
    {
        cout << title << " - " << artist << endl;
    }

    void play()
    {
        plays++;
    }
};
```

Now print and play are not external functions.

They are member functions, that belong to the song.

Using the class:

```
Song song1;  
  
song1.title = "Imagine";  
song1.artist = "John Lennon";  
song1.duration = 183;  
song1.plays = 0;  
  
song1.play();  
song1.print();
```

We are not asking an external function to print the song.

We are asking the song to print itself.

CLASS AND OBJECT

The words **class** and **object** are often confused.

A class is a general description.

An object is a specific instance created *from* that description.

For example, Song is a class.

```
class Song
{
    ...
};
```

But these are objects:

```
Song song1;
Song song2;
Song song3;
```

The class describes what every song has and what every song can do.

Each object stores its own data.

Changing song1 does not change song2.

MEMBERS OF A CLASS

Everything declared inside a class is called a member.

There are two main types of members.

DATA MEMBERS

Data members store the state of the object.

```
string title;  
string artist;  
int duration;  
int plays;
```

They answer the question: *What does the object know?*

MEMBER FUNCTIONS

Member functions define the behavior of the object.

```
void print();  
void play();  
void setDuration(int duration);
```

They answer the question: *What can the object do?*

DESIGN INSIGHT

When designing a class, ask two questions:

What does this object know?

What can this object do?

If the answers belong to the same idea, they probably belong in the same class.

THE MAIN PROBLEM STILL EXISTS

Our class is better than the original struct because it combines data and behavior.

But it still has a problem.

All data members are public.

That means we can still write:

```
song1.duration = -50;  
song1.plays = -100;
```

So we have not really protected the object yet.

To solve this, C++ gives us **access specifiers**.

ACCESS SPECIFIERS

Access specifiers control who can access the members of a class.

The two most important access specifiers are:

`public:`

and

`private:`

Public members can be accessed from outside the class.

Private members can only be accessed from inside the class.

PUBLIC VS. PRIVATE

Access level	Meaning
Public	Part of the external interface of the object.
Private	Internal implementation details of the object.

A good class usually keeps its data private and exposes only carefully selected public functions.

PROTECTING THE SONG – ENCAPSULATION IDEA

Let us redesign the class.

```
class Song
{
private:
    string title;
    string artist;
    int duration;
    int plays;

public:
    void print()    {
        cout << title << " - " << artist << endl;
    }
    void play()    {
        plays++;
    }
    void setDuration(int seconds)    {
        if (seconds > 0)
            duration = seconds;
    }
}
```

```
int getDuration()    {  
    return duration;  
}  
};
```

Now its no legal:

```
song1.duration = -50;
```

The compiler will reject it because duration is private.

Instead, we must write:

```
song1.setDuration(183);
```

The object controls how its own data changes.

This is the beginning of encapsulation.

ENCAPSULATION

Encapsulation means that an object *protects* its internal data and exposes only a controlled *interface*. Instead of allowing direct access to the data, the class provides functions that decide what is allowed. For example:

```
void setDuration(int seconds)
{
    if (seconds > 0)
        duration = seconds;
}
```

The object does not blindly accept every value. It protects itself.

A FIRST LOOK AT 'THIS'

```
void Song::setDuration(int duration)
{
    this->duration = duration;
}
```

The keyword 'this' is a pointer to the object that called the member function. For now, think of it simply as 'the current object'. We will study it in greater detail later in the course.

QUESTION

Why not make everything public? It is easier.

That is true. It is easier in the beginning. But in larger programs, making everything public creates serious problems. If every part of the program can change every field, then no object can protect itself. Professional software is not designed only to work today. It is designed to remain understandable and maintainable tomorrow.

STRUCT VS. CLASS

In C++, struct and class are very similar. Both can contain:

- data members,
- member functions, constructors, destructors, operators.

The main difference is the default access level.

Type	Default access level
Struct	Members are public by default.
Class	Members are private by default.

Because object-oriented programming emphasizes protection and encapsulation, we usually use class when designing objects.

A SIMPLE CLASS DECLARATION

Usually, we first write the class *declaration*.

```
class Song
{
private:
    string title;
    string artist;
    int duration;
    int plays;

public:
    void print();
    void play();
    void setDuration(int seconds);
    int getDuration();
};
```

This tells us **what** the class contains.

But the function bodies are not written here.

They will be written separately.

SEPARATING INTERFACE AND IMPLEMENTATION

In professional C++ programming, we usually separate a class into two files.

The header file contains the class declaration.

The source file contains the implementation.

For example:

- Song.h
- Song.cpp
- main.cpp

This helps organize the code and makes the project easier to maintain.

THE HEADER FILE: SONG.H

```
#ifndef SONG_H
#define SONG_H
#include <string>
using namespace std;
class Song
{
private:
    string title;
    string artist;
    int duration;
    int plays;

public:
    void print();
    void play();
    void setDuration(int seconds);
    int getDuration();
};

#endif
```

The header file describes **what** the class can do. It is the interface of the class.

THE SOURCE FILE: SONG.CPP

```
#include "Song.h"
#include <iostream>
using namespace std;

void Song::print() {
    cout << title << " - " << artist << endl;
}
void Song::play() {
    plays++;
}
void Song::setDuration(int seconds) {
    if (seconds > 0)    duration = seconds;
}
int Song::getDuration() {
    return duration;
}
```

The source file explains **how** the class works.

Notice the use of:

```
Song::
```

This tells the compiler that the function belongs to the Song class.

THE MAIN FILE: MAIN.CPP

```
#include "Song.h"

int main()
{
    Song song1;

    song1.setDuration(183);
    song1.play();
    song1.print();

    return 0;
}
```

The main function uses the class.

It does not need to know how play or print are implemented.

It only needs to know that these functions exist.

INTERFACE VS. IMPLEMENTATION

Term	Meaning
Interface	What the class allows other code to do.
Implementation	How the class performs those actions.

For example:

```
song1.play();
```

The user of the class does not need to know that the number of plays is updated.

The user only needs to know that calling `play()` plays the song.

This separation is one of the foundations of good software design.

IMPORTANT OBSERVATION

At this point, our class has a weakness.

We can create a song, but we have no good way to set its title or artist.

```
Song song1;
```

What is the title of song1? What is the artist? What is its duration?

What is its number of plays? We do not know.

This means the object may begin its life in an invalid or incomplete state.

This problem will lead us to constructors in the next unit.

SUMMARY

In this unit, we moved from struct to class.

We learned that:

- struct is useful for grouping related data.
- Public data can easily be changed incorrectly.
- Functions that always operate on the same data should often belong with that data.
- A class combines data and behavior.
- A class is a blueprint; an object is an instance of that blueprint.
- Data members describe what an object knows.
- Member functions describe what an object can do.
- public defines the external interface.
- private protects internal implementation details.
- Encapsulation allows objects to control their own data.
- Header files and source files help organize C++ projects.

LOOKING AHEAD

We now know how to **define** a class and **protect** its data.

And important question remains:

How do we make sure that an object starts its life in a valid state?

In the next unit, we will answer this question by introducing **constructors**.

Constructors are special functions that run when an object is created.

They allow us to initialize objects properly and continue building the idea that every object should be responsible for its own state.